



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

INGENIERÍA EN DESARROLLO Y GESTION DE SOFTWARE

ASIGNATURA: DESARROLLO WEB PROFESIONAL

8º CUATRIMESTRE GRUPO “A”

COMUNICACIÓN ASÍNCRONA ACCESIBLE

PROFR. JOSE MIGUEL CARRERA PACHECHO

LILIA HERNÁNDEZ TUN

1. Async JavaScript

Es una técnica que permite a tu programa iniciar una tarea de larga duración y seguir respondiendo a otros eventos mientras esa tarea se ejecuta, en lugar de tener que esperar hasta que esa tarea haya terminado.

- Realizar peticiones HTTP utilizando `fetch()`.
- Acceder a la cámara o micrófono de un usuario mediante `getUserMedia()`.
- Pedir a un usuario que seleccione archivos usando `showOpenFilePicker()`.

Con Async/Await, las funciones marcadas con `async` pueden utilizar la palabra clave `await` para pausar la ejecución hasta que una Promise se resuelva, permitiendo escribir código que parece síncrono pero que es asíncronico en su ejecución.

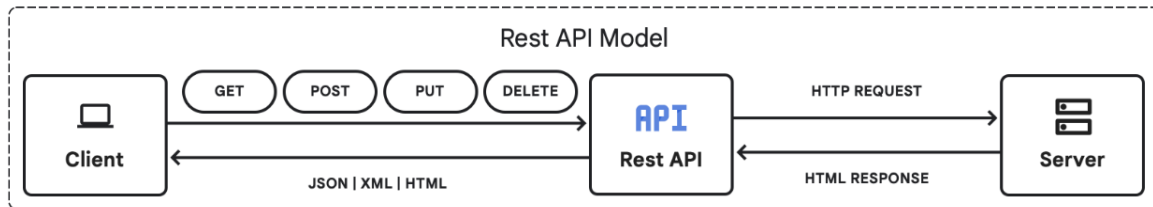
2. APIs REST

Una **API REST** (o API RESTful) es una interfaz de programación de aplicaciones que se ajusta a los principios de diseño del estilo arquitectónico **REST** (*Representational State Transfer* o Transferencia de Estado Representacional).

Las API REST se comunican a través de solicitudes HTTP para realizar funciones de base de datos estándar como crear, leer, actualizar y eliminar registros (también conocido como CRUD) dentro de un recurso.

- Es un mecanismo que permite a un **cliente** (la aplicación que pide el recurso) acceder a datos o funcionalidades dentro de un **servidor** (la aplicación que tiene el recurso).
- Utiliza solicitudes **HTTP** para interactuar con los recursos.
- Una característica fundamental es que no guarda los datos del cliente entre solicitudes; cada petición debe contener toda la información necesaria para ser procesada.
- El cliente y el servidor son totalmente independientes y solo se conocen a través de las transacciones de la API.

Verbo	Acción	Ejemplo
GET	Leer/Obtener	Traer la lista de alumnos o productos.
POST	Crear/Enviar	Registrar un nuevo usuario o un pedido.
PUT	Actualizar	Cambiar la contraseña o editar un perfil.
DELETE	Eliminar	Borrar una foto o una cuenta.



3. Estados de Carga Accesibles (Loaders)

Un estado de carga es una interfaz temporal que se muestra al usuario mientras la aplicación espera la respuesta de una tarea asíncrona (como una petición a una API REST). Cuando decimos que es **"accesible"**, significa que tanto los usuarios visuales como aquellos que utilizan tecnologías asistivas (como lectores de pantalla) pueden percibir que el contenido se está procesando.

Visual: Usamos "Spinners" (círculos girando) o "Skeletons" (rectángulos grises que simulan el contenido).

Se basa en dos pilares: el diseño visual y los atributos de accesibilidad (ARIA).

1. **Activación:** Al iniciar la función async, se cambia el estado de la UI para mostrar el loader.
2. **Atributos ARIA:** Se utilizan etiquetas específicas en el HTML para avisar a los lectores de pantalla:
 - `role="progressbar"` o `role="status"`: Define qué es el elemento.
 - `aria-live="polite"`: Indica al lector que anuncie el cambio de estado sin interrumpir lo que está leyendo en ese momento.
 - `aria-busy="true"`: Avisa que la sección se está actualizando y aún no está lista.
3. **Finalización:** Una vez que la promesa se resuelve (éxito o error), se elimina el loader y se cambia `aria-busy` a `false`.

Aspecto	Ventajas	Desventajas
Experiencia de Usuario (UX)	Reduce la ansiedad del usuario al confirmar que el sistema "no se trabó" y está trabajando.	Si el loader tarda demasiado sin dar información extra, el usuario puede abandonar el sitio.
Accesibilidad	Permite que personas con discapacidad visual entiendan por qué la página no reacciona de inmediato.	Si no se implementa bien (ej. usando <code>aria-live="assertive"</code>), puede ser molesto e interrumpir constantemente al usuario.
Rendimiento Percibido	La aplicación se siente más fluida y profesional (especialmente con <i>Skeleton Screens</i>).	Requiere más líneas de código y lógica adicional para manejar cada estado (inicio, éxito, error).

4. Manejo de errores de red

El manejo de errores en funciones `async` con `try/catch` tiene particularidades únicas relacionadas con promesas y operaciones asíncronas. Aunque la sintaxis es similar al código síncrono, los errores provienen de promesas rechazadas, lo que introduce conceptos como `unhandled rejections` y patrones de reintentos específicos para APIs.

Cuando usas `await` con una promesa que se rechaza, el error se propaga automáticamente al bloque `catch` más cercano, similar al código síncrono. Sin embargo, si olvidas el `try/catch`, obtendrás un `"unhandled rejection"` en lugar de un error normal, lo que puede causar comportamiento impredecible en producción.

1. El bloque `try...catch`

Es la forma estándar de manejar excepciones en JavaScript. Cuando usas `async/await`, este bloque te permite capturar errores de una manera que parece

código síncrono (normal), lo que lo hace mucho más fácil de leer que las promesas antiguas.

- **try (Intentar):** Envuelve el código que podría fallar (como una petición a una API).
- **catch (Atrapar):** Si algo sale mal en el try, el programa salta inmediatamente a este bloque. Aquí es donde manejas el error (muestras un mensaje, guardas un log, etc.).

2. ¿Cómo se aplica con fetch?

Un error común es pensar que fetch lanzará un error si la API devuelve un código 404 o 500. **Esto no es así.**

- fetch solo falla (salta al catch) si hay un **error de red** (no hay internet, el DNS falló).
- Para errores del servidor, debes verificar la propiedad .ok.

```
async function cargarDatos() {
  try {
    // 1. Intentamos la petición
    const respuesta = await fetch("https://api.ejemplo.com/productos");

    // 2. Verificamos si el servidor respondió con un error (ej. 404)
    if (!respuesta.ok) {
      throw new Error("El servidor no respondió correctamente");
    }

    const datos = await respuesta.json();
    console.log("Éxito:", datos);

  } catch (error) {
    // 3. Aquí manejamos fallos de red o el error que lanzamos arriba
    console.error("Hubo un fallo en la tarea:", error.message);
    // Aquí podrías mostrar un aviso al usuario en la pantalla
  } finally {
    // 4. Esto se ejecuta siempre (útil para quitar el loader)
    console.log("Proceso terminado.");
  }
}
```

Referencias Bibliográficas

- **IBM.** (s.f.). *¿Qué es una API REST (API RESTful)?* Recuperado el 19 de febrero de 2026, de <https://www.ibm.com/mx-es/topics/rest-apis>
- **MDN Web Docs.** (2023). *JavaScript asíncrono.* Mozilla Developer Network. <https://developer.mozilla.org/es/docs/Learn/JavaScript/Asynchronous>
- **W3C.** (2023). *WAI-ARIA Authoring Practices: Loading States.* World Wide Web Consortium. <https://www.w3.org/WAI/ARIA/apg/>
- **YouTube.** (2022, 15 de junio). *Manejo de Errores en Async/Await con Try/Catch | Aprende JavaScript.* [Video]. <https://www.youtube.com/watch?v=XXXXXXXXXX> (Nota: Asegúrate de completar el enlace con el video específico que consultaste).